Programação Aplicada em .Net

Aula 1

Prof. Me. Lucas Teodoro dos Santos

UNIP

24 de Agosto de 2026

Roteiro da Aula

1. Compilação e Execução de uma Aplicação
2. Reflection
3. Attributes

UNIP
UNIVERSIDADE PAULISTA

Compilação e Execução de uma Aplicação

Em uma aplicação C#, existem dois momentos importantes: a compilação e a execução.

- **Tempo de compilação (Compile Time)**: ocorre quando o código é analisado e convertido para uma forma que poderá ser executada;
- Nesse momento, o compilador identifica erros como variáveis inexistentes, tipos incompatíveis e problemas de sintaxe;
- **Tempo de execução (Run Time)**: ocorre quando o programa está efetivamente sendo executado pelo usuário;
- Alguns problemas só aparecem durante a execução, como divisão por zero, arquivo inexistente ou tentativa de acessar uma posição inválida de uma lista;
- Corrigir erros de compilação geralmente é mais simples, pois o Visual Studio costuma indicar a linha e o problema encontrado.

UNIP
UNIVERSIDADE PAULISTA

Exemplos de erros

Erro em tempo de compilação:

```
1           int idade = "20";
```

O código não é compilado, pois não é possível atribuir um texto a uma variável do tipo `int`.

Erro em tempo de execução:

```
1           Console.Write("Digite o numerador: ");
2           int numerador = int.Parse(Console.ReadLine());
3
4           Console.Write("Digite o denominador: ");
5           int denominador = int.Parse(Console.ReadLine());
6
7           int resultado = numerador / denominador;
8
9           Console.WriteLine($"Resultado: {resultado}");
```

O código compila normalmente. Porém, se o usuário informar `0` como denominador, ocorrerá um erro durante a execução.

Por que usar Reflection?

```
1           public static void ListarDados(object objeto)
2           {
3               switch (objeto)
4               {
5                   case Produto produto:
6                       Console.WriteLine($"Nome: {produto.Nome}");
7                       Console.WriteLine($"Preco: {produto.Preco:C}");
8                   break;
9
10                  case Cliente cliente:
11                      Console.WriteLine($"Nome: {cliente.Nome}");
12                      Console.WriteLine($"E-mail: {cliente.Email}");
13                  break;
14
15                  default:
16                      Console.WriteLine("Tipo nao reconhecido.");
17                  break;
18              }
19          }
```

UNIVERSIDADE PAULISTA

Por que usar Reflection?

O método anterior funciona apenas para os tipos que já foram previstos: `Produto` e `Cliente`.

Imagine que a aplicação passe a trabalhar também com uma classe `Funcionario`.

- Precisaremos alterar o método `ListarDados()`;
- Será necessário criar um novo `case` para a classe `Funcionario`;
- A cada novo tipo, o método deverá ser modificado novamente.

Princípio Open/Closed – SOLID

O código deve estar **aberto para extensão**, mas **fechado para modificação**.

Pergunta

Como listar os dados de qualquer objeto sem precisar conhecer previamente todas as classes da aplicação?

UNIVERSIDADE PAULISTA

O que é Reflection?

Reflection é um recurso do .NET que permite consultar informações sobre classes e objetos durante a execução da aplicação.

Com reflection, podemos analisar a estrutura de uma classe e identificar:

- O nome da classe;
- As propriedades disponíveis;
- Os métodos existentes;
- Os tipos das propriedades;
- Os construtores, campos e interfaces.

Ideia principal

Reflection permite que o programa conheça a estrutura de uma classe enquanto está sendo executado.

UNIVERSIDADE PAULISTA

O que é Type?

`Type` é uma classe do .NET utilizada para representar informações sobre um tipo, como uma classe, `string`, `int` ou `DateTime`.

No contexto de reflection, o `Type` permite que a aplicação "olhe" a estrutura de uma classe durante a execução.

- Nome da classe;
- Propriedades existentes;
- Métodos disponíveis;
- Tipos das propriedades.

Relação com Reflection

Reflection utiliza o `Type` para consultar informações sobre a estrutura de classes e objetos.

UNIP
UNIVERSIDADE PAULISTA

Classe utilizada nos exemplos

Nos exemplos a seguir, utilizaremos uma classe simples de produto.

```
1           public class Produto
2           {
3               public string Nome { get; set; }
4               public decimal Preco { get; set; }
5
6               public void ExibirResumo()
7               {
8                   Console.WriteLine($"{Nome} - {Preco:C}");
9               }
10          }
```

A classe possui duas propriedades e um método público.

Obtendo o Type de um objeto

Quando já possuímos um objeto instanciado, podemos utilizar o método `GetType()`.

```
1           Produto produto = new Produto
2           {
3               Nome = "Notebook",
4               Preco = 3500
5           };
6
7           Type tipo = produto.GetType();
8
9           Console.WriteLine(tipo.Name);
```

A saída será `Produto`.

Obtendo o Type sem criar um objeto

Quando queremos consultar a estrutura de uma classe sem instanciar um objeto, podemos utilizar `typeof`.

```
1           Type tipo = typeof(Produto);
2
3           Console.WriteLine(tipo.Name);
```

- `typeof(Produto)` obtém o Type da classe `Produto`;
- Não é necessário criar um objeto com `new Produto()`;
- É útil quando queremos analisar apenas a estrutura da classe.

UNIP
UNIVERSIDADE PAULISTA

Principais métodos de consulta

A partir de um objeto `Type`, podemos consultar diferentes elementos da estrutura de uma classe.

- `GetProperties()`: obtém as propriedades;
- `GetMethods()`: obtém os métodos;
- `GetConstructors()`: obtém os construtores;
- `GetFields()`: obtém os campos públicos;
- `GetInterfaces()`: obtém as interfaces implementadas;
- `GetCustomAttributes()`: obtém os attributes aplicados.

Nesta primeira abordagem, vamos focar principalmente em propriedades e métodos.

UNIP
UNIVERSIDADE PAULISTA

Consultando propriedades

O método `GetProperties()` permite obter todas as propriedades públicas de uma classe.

```
1           Type tipo = typeof(Produto);
2
3           foreach (var propriedade in tipo.GetProperties())
4           {
5               Console.WriteLine(
6               $"{propriedade.Name} - " +
7               $"{propriedade.PropertyType.Name}");
8           }
```

Saída:

- `Nome - String;`
- `Preco - Decimal.`

Consultando métodos

O método `GetMethods()` retorna os métodos públicos disponíveis em uma classe.

```
1           Type tipo = typeof(Produto);
2
3           foreach (var metodo in tipo.GetMethods())
4           {
5               Console.WriteLine(metodo.Name);
6           }
```

Entre os resultados, teremos o método `ExibirResumo`.

Observação: também poderão ser exibidos métodos herdados de `Object`, como `ToString()`, `Equals()` e `GetType()`.

Listando os dados de qualquer objeto

Podemos criar um método genérico que lista propriedades e valores sem conhecer previamente a classe recebida.

```
1           public static void ListarDados(object objeto)
2           {
3               Type tipo = objeto.GetType();
4
5               Console.WriteLine($"Classe: {tipo.Name}");
6
7               foreach (var propriedade in tipo.GetProperties())
8               {
9                   object valor = propriedade.GetValue(objeto);
10
11                  Console.WriteLine(
12                  $"{propriedade.Name}: {valor}");
13              }
14          }
```

Utilizando o método genérico

O método `ListarDados()` pode receber objetos de diferentes classes.

```
1           Produto produto = new Produto
2           {
3               Nome = "Notebook",
4               Preco = 3500
5           };
6
7           ListarDados(produto);
```

Saída:

```
1           Classe: Produto
2           Nome: Notebook
3           Preco: 3500
```

Aplicações práticas

Reflection é especialmente útil para criar recursos genéricos, que podem trabalhar com diferentes classes.

- Geração de logs e relatórios;
- Ferramentas de diagnóstico;
- Validações genéricas;
- Serialização de objetos;
- Frameworks, como ASP.NET Core e Entity Framework Core.

Exemplo

O método `ListarDados()` pode ser utilizado como apoio para exibir informações de objetos durante o desenvolvimento e a depuração da aplicação.

UNIP
UNIVERSIDADE PAULISTA

Cuidados ao utilizar Reflection

- Reflection é útil quando precisamos trabalhar com estruturas dinâmicas;
- Para acessos simples e conhecidos, prefira utilizar as propriedades e os métodos diretamente;
- O uso excessivo pode tornar o código mais complexo;
- Reflection pode ser mais lento que o acesso direto aos membros de uma classe.

Resumo

Utilize reflection quando o benefício de trabalhar de forma genérica for maior que a complexidade adicional do código.

UNIP
UNIVERSIDADE PAULISTA

O que são Attributes?

Attributes são informações adicionais que podem ser aplicadas a elementos do código, como classes, propriedades, métodos e parâmetros.

Eles funcionam como **metadados**: descrevem uma informação sobre o código, sem representar diretamente a regra de negócio da classe.

- Regras de validação;
- Nomes e descrições para propriedades;
- Configurações utilizadas por frameworks;
- Informações personalizadas para a aplicação.

UNIP
UNIVERSIDADE PAULISTA

O que são metadados?

Metadados são informações que descrevem outros dados.

No código, eles adicionam informações sobre classes, propriedades ou métodos, sem alterar diretamente os valores ou a lógica principal da aplicação.

Exemplo fora da programação

Em uma fotografia, a imagem é o dado. Informações como data, horário, localização e modelo da câmera são os metadados.

Em C#, os attributes são uma forma de adicionar metadados ao código.

- `[Required]` informa que uma propriedade é obrigatória;
- `[StringLength(100)]` informa o tamanho máximo permitido;
- `[Obsolete]` informa que um método não deve mais ser utilizado.

UNIP
UNIVERSIDADE PAULISTA

Onde os Attributes podem ser aplicados?

Attributes podem ser utilizados em diferentes partes do código.

- **Classes**: para adicionar informações sobre uma classe;
- **Propriedades**: para indicar validações ou descrições;
- **Métodos**: para indicar comportamentos específicos;
- **Parâmetros**: para descrever regras sobre valores recebidos.

Sintaxe

Os attributes são escritos entre colchetes, antes do elemento ao qual serão aplicados.

UNIP
UNIVERSIDADE PAULISTA

Exemplo simples

O attribute `Obsolete` indica que um método não deve mais ser utilizado.

```
public class Calculadora
{
    [Obsolete("Use CalcularTotalComDesconto.")]
    public decimal CalcularTotal(decimal valor)
    {
        return valor;
    }

    public decimal CalcularTotalComDesconto(
    decimal valor)
    {
        return valor * 0.9m;
    }
}
```

Quando outro desenvolvedor utilizar `CalcularTotal()`, o compilador exibirá um aviso com a mensagem informada.

Attributes no .NET e em frameworks

Attributes são utilizados pelo .NET e por diversos frameworks para adicionar configurações e comportamentos ao código.

- **Data Annotations**: `[Required]`, `[Range]` e `[StringLength]` para validações;
- **ASP.NET Core**: utiliza attributes para validar dados enviados em formulários e requisições;
- **Entity Framework Core**: utiliza attributes como `[Key]`, `[Table]` e `[NotMapped]` para mapeamento;
- **System.Text.Json**: utiliza `[JsonIgnore]` e `[JsonPropertyName]` para controlar a serialização em JSON;
- **Compilador**: utiliza `[Obsolete]` para indicar elementos que não devem mais ser utilizados.

UNIP
UNIVERSIDADE PAULISTA

Exemplo: Validação de propriedades

Os attributes de validação estão disponíveis no namespace `System.ComponentModel.DataAnnotations`.

```
using System.ComponentModel.DataAnnotations;

public class Produto
{
    [Required]
    [StringLength(100)]
    public string Nome { get; set; }

    [Range(0.01, 100000)]
    public decimal Preco { get; set; }
}
```

- `Nome` deve ser preenchido e ter até 100 caracteres;
- `Preco` deve estar entre 0,01 e 100.000.

Exemplo: Attributes no Entity Framework Core

No Entity Framework Core, attributes podem auxiliar no mapeamento entre classes e tabelas do banco de dados.

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

[Table("produtos")]
public class Produto
{
    [Key]
    public int Id { get; set; }

    [Column("nome_produto")]
    public string Nome { get; set; }

    public decimal Preco { get; set; }

    [NotMapped]
    public decimal PrecoComDesconto => Preco * 0.9m;
}
```

Como os Attributes funcionam?

O attribute apenas armazena uma informação no código.

Para que uma regra seja aplicada, algum código ou framework precisa interpretar essa informação.

Exemplo

O attribute `[Required]` informa que uma propriedade é obrigatória. No ASP.NET Core, o mecanismo de validação pode ler esse attribute e informar ao usuário caso o campo não seja preenchido.

Criando o Attribute `Exibir`

Criaremos um attribute para marcar quais propriedades podem ser exibidas por um método genérico.

```
1           [AttributeUsage(AttributeTargets.Property)]
2           public class ExibirAttribute : Attribute
3           {
4           }
```

Esse attribute não precisa armazenar informações. Sua presença na propriedade já será suficiente para indicar que ela deve ser exibida.

Regra de utilização

`AttributeUsage` define onde um attribute pode ser aplicado. Nesse caso, `AttributeTargets.Property` indica que `Exibir` só poderá ser utilizado em propriedades.

Utilizando o Attribute

```
1           public class Produto
2           {
3               [Exibir]
4               public string Nome { get; set; }
5
6               [Exibir]
7               public decimal Preco { get; set; }
8
9               public int Estoque { get; set; }
10          }
```

Nesse exemplo, o relatório poderá exibir `Nome` e `Preco`. A propriedade `Estoque` não possui o attribute.

Attributes como critério para Reflection

O attribute `[Exibir]` apenas adiciona uma informação à propriedade: ela pode ser exibida por determinado recurso.

Com reflection, podemos consultar as propriedades do objeto e decidir como tratá-las a partir dessa informação.

Duas abordagens

Na **reflection aberta**, todas as propriedades encontradas são utilizadas. Na **reflection controlada**, somente as propriedades marcadas com `[Exibir]` são utilizadas.

UNIP
UNIVERSIDADE PAULISTA

Reflection Aberta

Na reflection aberta, o método percorre todas as propriedades públicas encontradas no objeto.

```
1           public static void ListarDados(object objeto)
2           {
3               foreach (var propriedade in
4               objeto.GetType().GetProperties())
5               {
6                   Console.WriteLine(
7                   $"{propriedade.Name}: " +
8                   $"{propriedade.GetValue(objeto)}");
9               }
10          }
```

Para um `Produto`, seriam exibidos `Nome`, `Preco` e `Estoque`.

Reflection Controlada

Com o attribute [Exibir], o método considera apenas as propriedades marcadas explicitamente.

```
using System.Reflection;

public static void ListarDados(object objeto)
{
    foreach (var propriedade in
    objeto.GetType().GetProperties())
    {
        bool deveExibir = propriedade.GetCustomAttribute<
        ExibirAttribute>() != null;

        if (deveExibir)
        {
            Console.WriteLine(
            $"{propriedade.Name}: " +
            $"{propriedade.GetValue(objeto)}");
        }
    }
}
```

Reflection Aberta e Controlada

Aberta	**Controlada**
Percorre todas as propriedades públicas.	Percorre apenas propriedades marcadas com `[Exibir]`.
Não diferencia os dados relevantes.	A classe define explicitamente os dados que disponibiliza.
É mais simples, mas oferece menos controle.	Utiliza metadados para tornar o comportamento mais previsível.

Ligação entre os conceitos

O attribute define quais propriedades devem ser utilizadas. Reflection lê essa informação durante a execução e aplica o comportamento definido.

UNIP
UNIVERSIDADE PAULISTA

Resumo geral da aula

- Erros podem ocorrer em **tempo de compilação** ou em **tempo de execução**;
- Reflection permite consultar a estrutura de classes e objetos durante a execução;
- O objeto `Type` representa informações como nome, propriedades e métodos de uma classe;
- Com reflection, é possível criar recursos genéricos que trabalham com diferentes tipos de objetos;
- Attributes adicionam metadados ao código e são utilizados pelo .NET, ASP.NET Core e Entity Framework Core;
- Na reflection controlada, attributes definem quais propriedades serão utilizadas pelo código genérico.

Conexão principal

Attributes informam **o que** deve ser considerado. Reflection permite que a aplicação encontre e utilize essa informação durante a execução.

Exercício de fixação

Crie uma aplicação Console para exibir informações de objetos utilizando reflection e attributes.

1. Crie o attribute personalizado:

```
1           [AttributeUsage(AttributeTargets.Property)]
2           public class ExibirAttribute : Attribute
3           {
4           }
```

2. Crie duas classes: `Produto` e `Cliente`.

- `Produto`: `Nome`, `Preco` e `Estoque`;
- `Cliente`: `Nome`, `Email` e `Telefone`.

Marque apenas duas propriedades de cada classe com `[Exibir]`.

UNIP
UNIVERSIDADE PAULISTA

Exercício de fixação

3. Crie o método genérico abaixo:

```
1           public static void ListarDados(object objeto)
2           {
3               // Implementar utilizando reflection
4           }
```

O método deve:

1. Obter o `Type` do objeto recebido;
2. Percorrer suas propriedades públicas;
3. Verificar se cada propriedade possui o attribute `[Exibir]`;
4. Exibir o nome e o valor apenas das propriedades marcadas.

4. Instancie um `Produto` e um `Cliente`, chamando `ListarDados()` para os dois objetos.

Obrigado!

Dúvidas?

UNIP
UNIVERSIDADE PAULISTA